

ADVANCED COMPUTER NETWORKS

MODULE II

- **TRANSPORT LAYER**
- **SERVICES PROVIDED BY THE TRANSPORT LAYER**
- **TRANSPORT LAYER PROTOCOLS**
 - **TCP**
 - **UDP**
- **PRINCIPLES OF RELIABLE DATA TRANSFER**
 - **RDТ 1.0**
 - **RDТ 2.0**
 - **RDТ 3.0**
- **STOP AND WAIT PROTOCOL**
- **GO-BACK-N ARQ**
- **MULTIPLEXING**
 - **FDM**
 - **WDM**
 - **TDM:ASYNCHRONOUS & SYNCHRONOUS**

Transport Layer

Transport Layer is **Layer 4**.

It sits between:

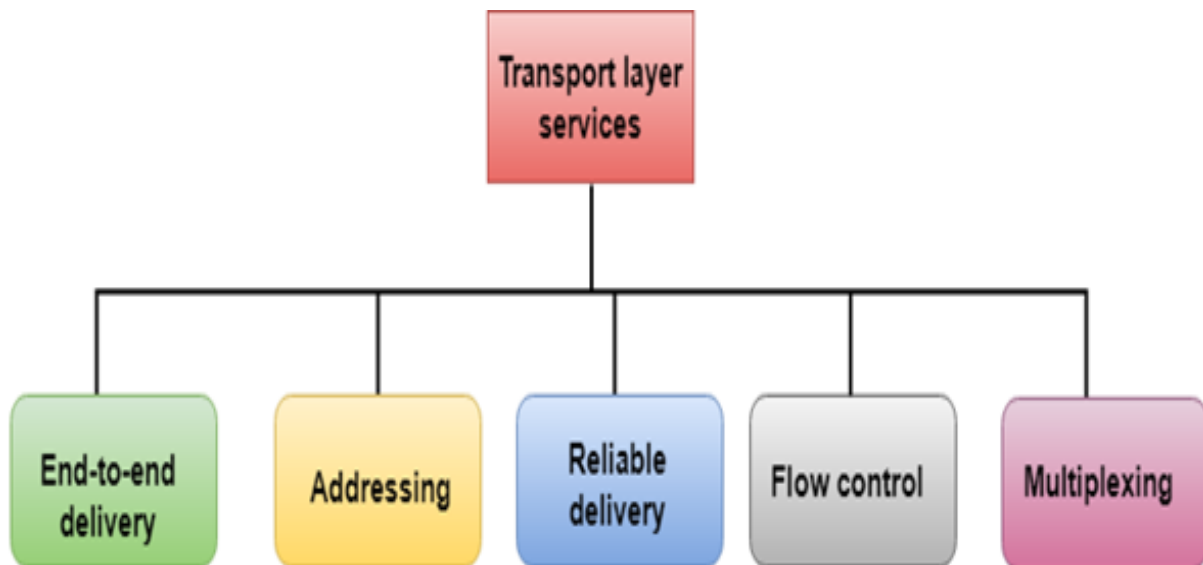
- **Network Layer (Layer 3)**
- **Session Layer (Layer 5)**

It provides **end-to-end communication** between applications

Main transport protocols:

- **TCP (Transmission Control Protocol)** → reliable communication
- **UDP (User Datagram Protocol)** → faster but unreliable communication

Services Provided by Transport Layer



1. End-to-End Delivery

- Sends complete data from source application → destination application.
- Ensures full message reaches the destination.

Example: Email sent from your laptop reaches another person's laptop.

2. Reliable Delivery

Ensures data reaches correctly without errors or loss.

It includes 4 controls:

(a) Error Control

- Detects and corrects transmission errors.
- Ensures data arrives correctly.
- Done end-to-end (not just between nodes like Data Link Layer).

Why needed?

- Errors may occur inside routers.
- Transport layer checks entire path.

(b) Sequence Control

- Data is divided into segments.
- Each segment gets a sequence number.
- Receiver reassembles data in correct order.

Prevents:

- Wrong order delivery.

(c) Loss Control

- Detects missing data.
- Retransmits lost segments.

Uses sequence numbers to find missing parts.

(d) Duplication Control

- Prevents duplicate data.
- Removes repeated segments.

Example: If same packet arrives twice → discard one.

3. Flow Control

- Prevents sender from sending data too fast.
- Protects receiver from overload.
- Uses **Sliding Window Protocol**.

Sliding Window:

- Controls how much data can be sent before acknowledgment.
- Makes transmission efficient.

4. Multiplexing

Improves transmission efficiency by managing multiple connections.

Types:

Upward Multiplexing

- Multiple transport connections use one network connection.
- Reduces cost and improves efficiency.

Example: Many apps using same internet connection.

Downward Multiplexing

- One transport connection uses multiple network paths.
- Improves speed and throughput.

Used when network is slow.

5. Addressing (Port Addressing)

- Ensures data goes to correct application on destination.
- Uses **port numbers**.

Important term:

- **TSAP (Transport Service Access Point)** → port number identifying application.

Example:

- Web browser → Port 80
- Email → Port 25

Transport Layer Protocols

- The transport layer mainly uses **two protocols**:
 - **TCP (Transmission Control Protocol)**
 - **UDP (User Datagram Protocol)**
- **IP protocol (Network Layer)** → delivers data **host to host**.
- **Transport layer protocols** → deliver data **process to process (port to port)**.

Port Numbers

- Each application uses a **port number**.
- Port number = **16-bit positive integer**.
- Used to identify applications.

Example:

- Web service → Port 80
- Email → Port 25

UDP (User Datagram Protocol)

- UDP = **User Datagram Protocol**
- Simple and fast protocol.
- **Connectionless protocol** (no connection setup).
- Used when **speed is more important than reliability**.

Example uses:

- Video streaming

- Online games
- Voice calls

Characteristics of UDP

- No connection establishment.
- No sequencing of packets.
- No retransmission.
- Faster transmission.
- Low overhead.
- Adds:
 - Port address
 - Length
 - Checksum error detection

Data unit of UDP → **User Datagram**

UDP Header Format (16 bytes)

Source port address 16 bits	Destination port address 16 bits
Total Length 16 bits	Checksum 16 bits
Data	

Fields:

1. **Source Port Address (16 bits)**
 - Sender application address.
2. **Destination Port Address (16 bits)**
 - Receiver application address.
3. **Total Length (16 bits)**
 - Total size of datagram.
4. **Checksum (16 bits)**
 - Error detection.

Disadvantages of UDP

- No reliability.
- No sequencing.
- No retransmission.
- Cannot detect lost packet.
- No duplicate removal.

It only detects error but does not correct it.

TCP (Transmission Control Protocol)

- TCP = **Transmission Control Protocol**
- Provides full transport services.
- **Connection-oriented protocol.**
- Creates a virtual connection before transmission.

Used when reliability is important.

Examples:

- Email
- Web browsing
- File transfer

Features of TCP

1. Stream Data Transfer

- Data sent as continuous stream of bytes.
- Divides data into segments.

2. Reliability

- Each byte has sequence number.
- Receiver sends acknowledgment (ACK).
- Lost data retransmitted.

3. Flow Control

- Prevents receiver overload.
- Uses **window mechanism**.

4. Multiplexing

- Data from different applications sent simultaneously.
- Receiver sends data to correct application.

5. Logical Connection

- Connection identified using sockets, sequence numbers, window size.

6. Full Duplex Communication

- Data flows in both directions simultaneously.

TCP Communication Steps:

1. Connection establishment.
2. Data transfer.
3. Connection termination.

TCP Segment Format

Source port address 16 bits							Destination port address 16 bits						
Sequence number 32 bits													
Acknowledgement number 32 bits													
HLEN 4 bits	Reserved 6 bits	URG	ACK	PSH	RST	SYN	FIN	Window size 16 bits					
Checksum 16 bits								Urgent pointer 16 bits					
Options & padding													

Fields

1. Source Port (16 bits)

- Sender application.

2. Destination Port (16 bits)

- Receiver application.

3. Sequence Number (32 bits)

- Position of data in stream.

4. Acknowledgement Number (32 bits)

- Next expected data.

5. Header Length

- Size of TCP header (20–60 bytes).

6. Control Flags

Flag	Function
URG	Urgent data
ACK	Acknowledgement valid
PSH	Push data immediately
RST	Reset connection
SYN	Start connection
FIN	End connection

7. Window Size

- Amount of data receiver can accept.

8. Checksum

- Error detection.

9. Urgent Pointer

- Points urgent data.

10. Options & Padding

- Extra information.

Reliable Data Transfer

Reliable Data Transfer ensures that data is delivered from sender to receiver correctly, completely, and in proper order without loss, duplication, or errors.

It is mainly provided by **TCP protocol** in the transport layer.

Why Reliable Data Transfer is needed

In real networks:

- Packets may be **lost**
- Data may be **corrupted**
- Packets may arrive in **wrong order**
- Duplicate packets may occur
- Receiver may be overloaded

Reliable data transfer solves these problems.

Goals of Reliable Data Transfer

Reliable data transfer ensures:

- ☐ No data loss
- ☐ No data corruption
- ☐ Correct order delivery
- ☐ No duplication
- ☐ Proper flow of data

Mechanisms Used in Reliable Data Transfer

Transport layer achieves reliability using the following techniques:

1. Error Detection

- Checks whether data is corrupted during transmission.
- Uses **checksum** method.
- If error detected → receiver informs sender.

Example:

If a bit changes from 0 → 1 during transmission → error detected.

2. Acknowledgement (ACK)

- Receiver sends confirmation after receiving data.
- Sender sends next data only after receiving ACK.

Types:

- **Positive ACK** → Data received correctly.
- **Negative ACK (NAK)** → Error detected.

Ensures reliable communication.

3. Retransmission

- If ACK not received within time → sender retransmits data.
- Helps recover lost packets.

Used in TCP.

4. Sequence Numbers

- Each packet gets a unique number.
- Helps:
 - Detect missing packets
 - Maintain order
 - Remove duplicates

Example:

Packet 1, 2, 3 → if 2 missing → request resend.

5. Flow Control

- Prevents sender from sending too much data.
- Avoids receiver overload.
- Uses **sliding window protocol**.

6. Timers

- Sender waits for ACK for specific time.
- If time expires → retransmission occurs.

Reliable Data Transfer that is, it assumes that the underlying channel has:

- No packet loss

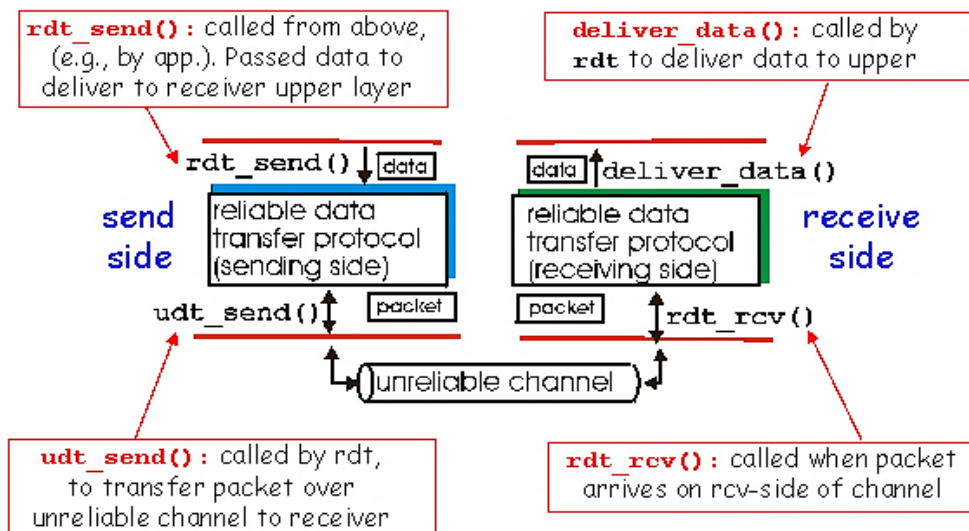
- No bit Corruption
- Synchronized



- Application layer is responsible for Sending and receiving process
- Transport layer provide reliable channel
- In transport layer there are 4 operations

- rdt_send()
 - udt_send()
 - rdt_rcv()
 - deliver_data()
- Sender Side
- Receiver Side

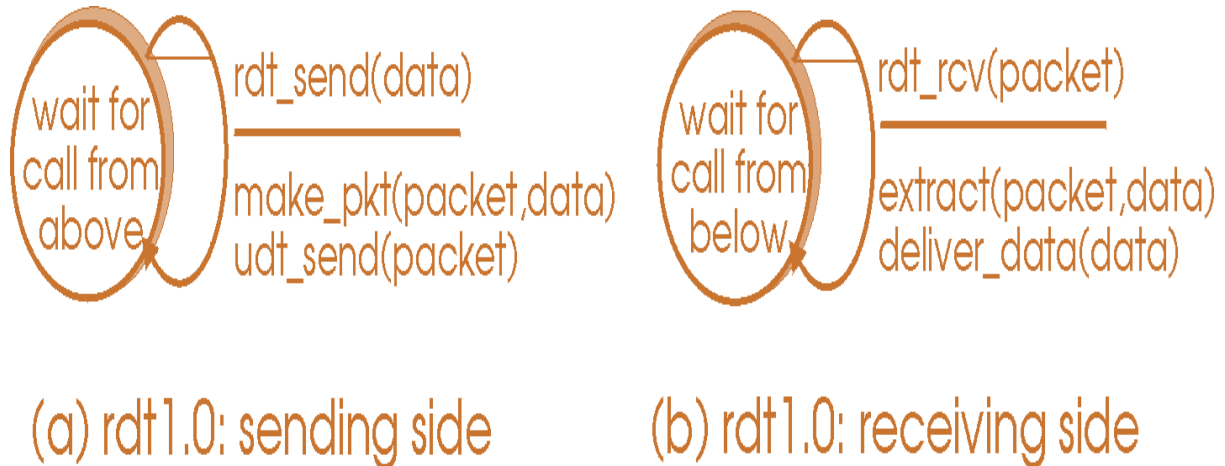
rdt: reliable data transfer
udt: unreliable data transfer
rcv: receive



Building a Reliable Data Transfer Protocol

- Reliable Data Transfer over a Perfectly Reliable Channel: **rdt1.0**
- Reliable Data Transfer over a Channel with Bit Errors: **rdt2.0**
- Reliable Data Transfer over a Lossy Channel with Bit Errors: **rdt3.0**

rdt1.0



- Use **finite state machine** (FSM) to specify sender and receiver
- FSM consists of a finite number of states each representing a specific condition or situation
- The sender and receiver FSMs, each have just **one state**.
- The arrows in the FSM description indicate the **transition of the protocol** from one state to another.
- The event causing the transition is shown above the horizontal line **labelling the transition**
- The **action(s) taken** when the event occurs are shown below the horizontal line.
- **Sending side**
 - accepts data from the upper-layer (via the `rdt_send(data)`),
 - puts the data into a packet (via the action `make_pkt(packet, data)`)
 - sends the packet into the channel (via the action `udt_send(packet)`).
- **Receiving side:**
 - receives a packet from the underlying channel via the `rdt_rcv(packet)`
 - Extract the data from the packet (via the action `extract(packet, data)`)
 - deliver the data up to the upper-layer via `deliver_data(data)`

- In this simple protocol, there is no difference between a unit of data and a packet. Also, all packet flow is from the sender to receiver - with a **perfectly reliable channel** there is **no need for the receiver side to provide any feedback to the sender** since nothing can go wrong!

rdt2.0(stop-and-wait)

Fundamentally, two additional protocol capabilities are required to handle the presence of bit errors:

- **Error detection:** First, a mechanism is needed to allow the receiver to detect when bit errors have occurred. uses the **checksum field** for exactly this purpose. These techniques allow the receiver to detect, and possibly correct packet bit errors.
- **Receiver feedback:** If the receiver detects an error, it can send a message back to the sender requesting retransmission of the corrupted data, enabling a mechanism for error correction.
 - ACK
 - NACK(Negative Acknowledgment)



(a) rdt2.0: sending side



(b) rdt2.0: receiving side

Sender Side

- It has two states
- In **one state**, is **waiting for data** from the upper layer.
- In the **other state**, **waiting for an ACK or a NACK** from the receiver
 - accepts data from the upper-layer (via the `rdt_send(data)`)

- Detect error (via compute checksum)
- Create packet (via the action make_pkt(packet,data))
- sends the packet into the channel (via the action udt_send(packet)).
- If the packet arrived with proper acknowledgement(isACK) then no action occurred
 - rdt_rcv(pkt) && isACK(rcvpkt)
- If the pkt arrived with error(isNACK), the sender retransmit the last packet using udt_send(pkt)
 - rdt_rcv(pkt) && isNACK(rcvpkt)
 udt_send(pkt)
- Thus, the sender will not send a new piece of data until it is sure that the receiver has correctly received the current packet. Because of this behaviour, protocols such as rdt2.0 are known as **stop-and-wait** protocols.

receiver-side

- ❖ The receiver-side FSM for rdt2.0 still has a single state.
- ❖ On packet arrival, the receiver replies with either an ACK or a NACK, depending on whether or not the received packet is corrupted.
- ❖ If the packet is received with error free (**not corrupted**) then **extract** the received packet and **deliver** the data to upper layer and also send the **acknowledgment** to sender

rdt_rcv(rcvpkt) && notcorrupt(rcvpkt)

extract(pkt,data)

deliver_data(data)

udt_send(ACK)

- ❖ If the received packet have error (**corrupted**), receiver will send the **negative acknowledgment** to the sender. Then the sender will **retransmit** the packet

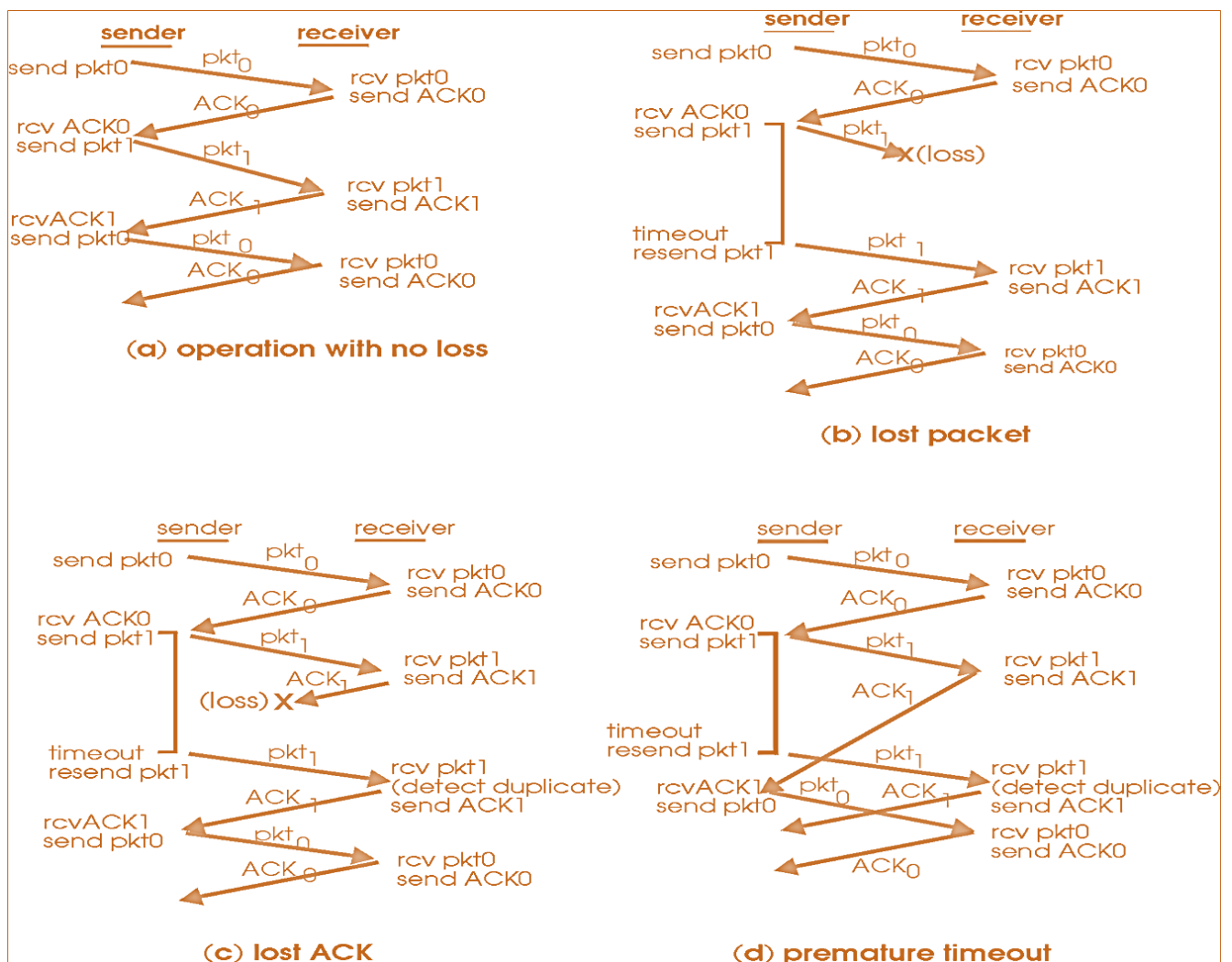
rdt_rcv(rcvpkt) && corrupt(rcvpkt)

udt_send(NACK)

rdt 3.0

- Underlying channel can also lose packets(data,ACKs).

- Even checksum, Sequence, ACKs, Retransmission will not help
- Sender waits “**reasonable**” amount of time for ACK
- It retransmit if no ACK received in this time.
- If packet(or ACK) delayed(not lost):
 - Retransmission will be updated, but sequence already handle it
- Receiver must specify sequence of packet being ACKed
- It requires countdown timer.



Stop-and-Wait protocol

Stop-and-Wait protocol is a reliable data transfer technique in which the sender sends one data frame at a time and waits for an acknowledgement (ACK) from the receiver before sending the next frame.

- It ensures **error-free and reliable transmission**.

- It is the simplest **flow control and error control protocol**.
- Used in **transport layer and data link layer**.

Why Stop-and-Wait Protocol is Needed

In data transmission:

- Data may be lost.
- Receiver may be overloaded.
- Errors may occur.
- Data may arrive out of order.

Stop-and-Wait prevents these problems by sending data slowly and carefully.

Working of Stop-and-Wait Protocol (Step-by-Step)

Step 1: Sender Sends Frame

- Sender transmits one data frame.

Step 2: Sender Stops and Waits

- Sender waits for acknowledgement (ACK).

Step 3: Receiver Checks Data

- If data is correct → sends ACK.
- If data has error → no ACK or sends NAK.

Step 4: Sender Receives ACK

- If ACK received → send next frame.
- If no ACK → resend frame.

Process repeats until all data is sent.

Example

Suppose sender wants to send 3 frames:

1. Send Frame 1 → Wait → ACK received
2. Send Frame 2 → Wait → ACK received
3. Send Frame 3 → Wait → ACK received

Only one frame sent at a time.

Important Features

1. Flow Control

- Prevents receiver overload.
- Sender sends next data only after ACK.

2. Error Control

- Lost or damaged frames are retransmitted.

3. Sequence Numbers

- Uses 0 and 1 sequence numbers.
- Prevents duplicate frames.

Use of Timer (Timeout Mechanism)

- Sender starts timer after sending frame.
- If ACK not received within time → retransmission occurs.

Solves problem of lost frames or lost ACK.

Disadvantages of Stop and Wait protocol

1. Problems occur due to lost data

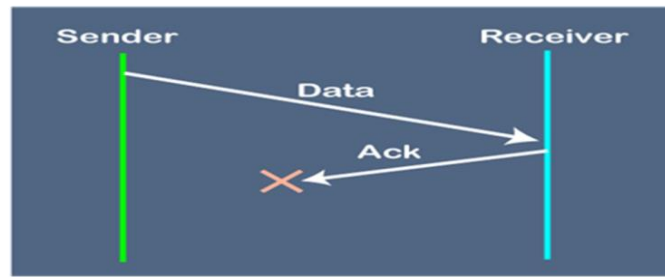


In this case, two problems occur:

Sender waits for an infinite amount of time for an acknowledgment.

Receiver waits for an infinite amount of time for a data.

2. Problems occur due to lost acknowledgment



In this case, one problem occurs:

Sender waits for an infinite amount of time for an acknowledgment.

3. Problem due to the delayed data or acknowledgment



Suppose the sender sends the data and it has also been received by the receiver.

The receiver then sends the acknowledgment but the acknowledgment is received after the timeout period on the sender's side.

As the acknowledgment is received late, so acknowledgment can be wrongly considered as the acknowledgment of some other data packet.

Efficiency Calculation

A channel has:

- Data frame size = **1000 bits**
- Bandwidth = **1 Mbps**
- Propagation delay = **20 ms**
- ACK size = negligible

Find the efficiency of Stop-and-Wait protocol.

Efficiency Formula

$$Efficiency = \frac{Transmission\ time}{Total\ time}$$

Step 1: Transmission Time (Tt)

$$Tt = \frac{\text{Frame size}}{\text{Bandwidth}}$$
$$Tt = \frac{1000}{1,000,000} = 0.001 \text{ sec} = 1 \text{ ms}$$

Step 2: Total Time

$$\begin{aligned} \text{Total Time} &= Tt + 2 \times \text{Propagation delay} \\ &= 1 + 2(20) \\ &= 1 + 40 = 41 \text{ ms} \end{aligned}$$

Step 3: Efficiency Formula

$$\begin{aligned} \text{Efficiency} &= \frac{\text{Transmission time}}{\text{Total time}} \\ &= \frac{1}{41} \\ &= 0.024 = 2.4\% \end{aligned}$$

Efficiency = 2.4% (Very Low)

Shows Stop-and-Wait is inefficient for long propagation delay.

Go-Back-N ARQ Protocol (Automatic Repeat Request)

Go-Back-N ARQ is a reliable data transfer protocol in which the sender can send multiple frames continuously without waiting for acknowledgement, but if an error occurs, all frames from the error point are retransmitted.

- Uses **sliding window mechanism**.
- Improves efficiency compared to Stop-and-Wait.
- Used for **flow control and error control**.

Why Go-Back-N is Needed

Stop-and-Wait protocol is slow because:

- Sender sends only one frame at a time.
- Channel remains idle while waiting.

Go-Back-N solves this by allowing **multiple frames to be sent at once**.

- Sender sends **N frames continuously**.
- Receiver accepts frames **in order only**.

- If one frame is lost → receiver rejects all later frames.
- Sender retransmits from the lost frame onward.

That's why called **Go-Back-N**.

Working of Go-Back-N Protocol

Step 1: Window Size

- Sender has a window of size **N**.
- Can send N frames without waiting.

Step 2: Continuous Transmission

- Sender sends frames 0, 1, 2, 3... continuously.

Step 3: Receiver Checks Frames

- If frame received correctly → send ACK.
- If frame missing → discard later frames.

Step 4: Error Handling

- Sender retransmits lost frame and all following frames.

Example

Suppose window size = 4

Sender sends:

- Frame 0 ✓
- Frame 1 ✓
- Frame 2 ☐ (lost)
- Frame 3 (discarded by receiver)

Receiver sends ACK for frame 1 only.

Sender retransmits:

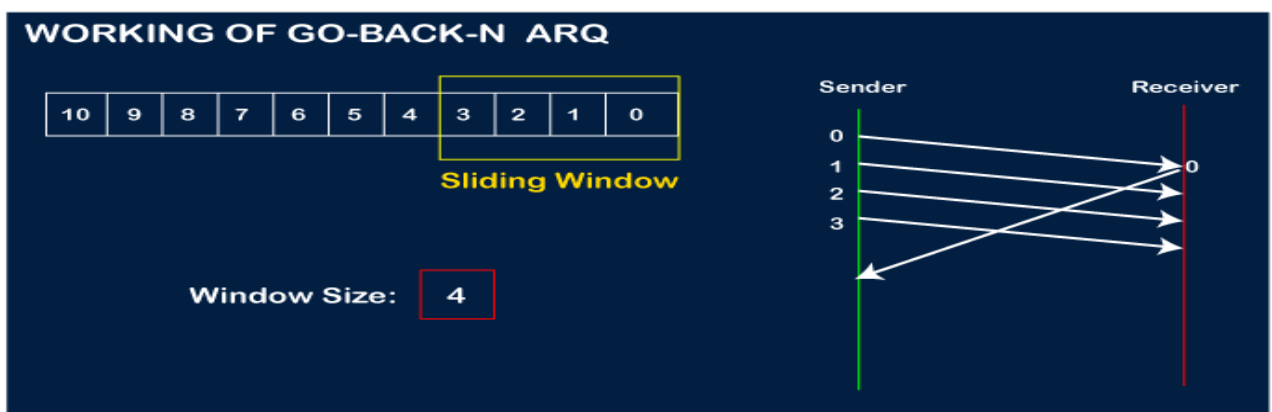
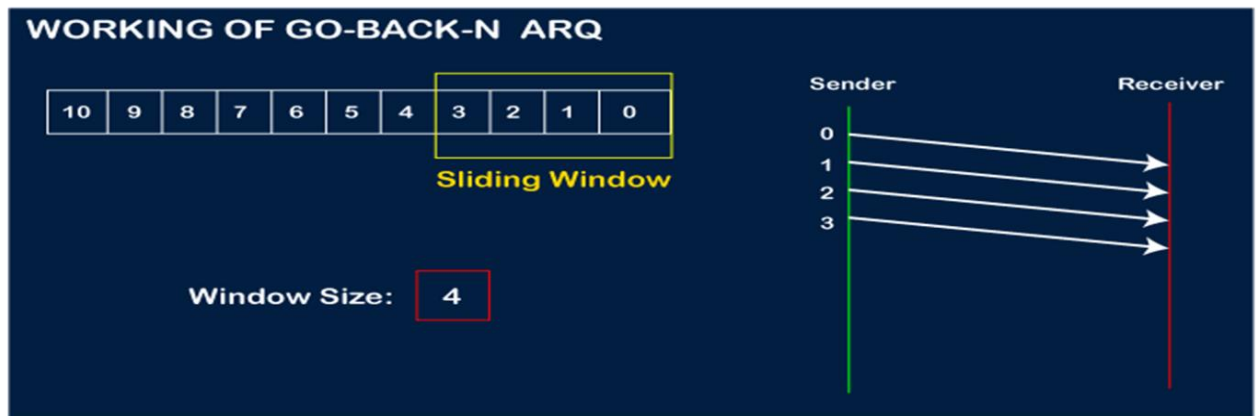
- Frame 2
- Frame 3

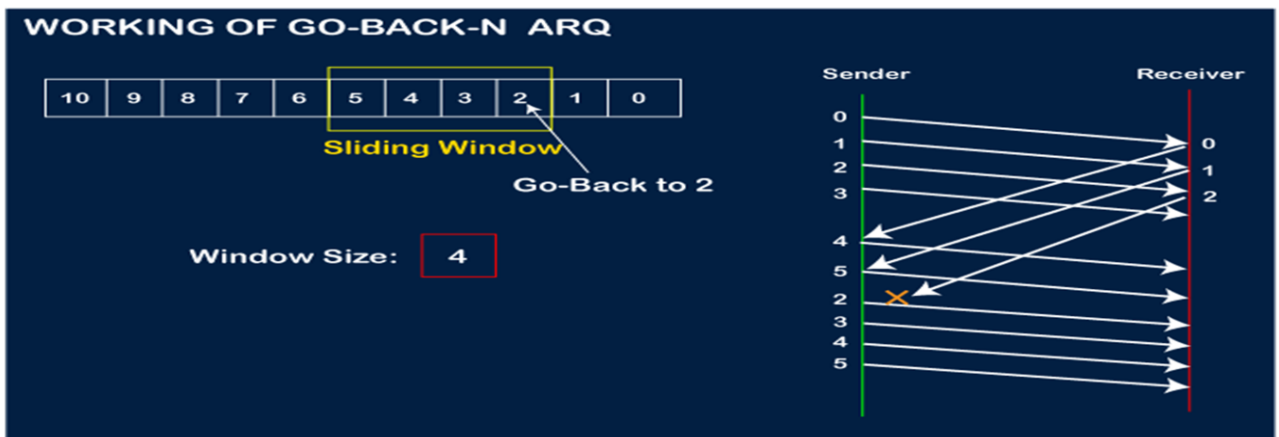
Working of Go-Back-N ARQ

- Suppose there are a **sender** and a **receiver**, and let's assume that there are **11 frames** to be sent.
- These frames are represented as **0,1,2,3,4,5,6,7,8,9,10**, and these are the **sequence numbers** of the frames.
- Mainly, the sequence number is decided by the **sender's window size**.

- But, for the better understanding, we took the running sequence numbers, i.e., 0,1,2,3,4,5,6,7,8,9,10.

Let's consider the window size as 4, which means that the four frames can be sent at a time before expecting the acknowledgment of the first frame.





PROBLEM I

Host A wants to send 10 frames to Host B. How many frames are transmitted by Host A if every 6th frame transmitted by Host A is either corrupted or lost? What would be the number of transmissions in Go-Back-4 ARQ?

PROBLEM II

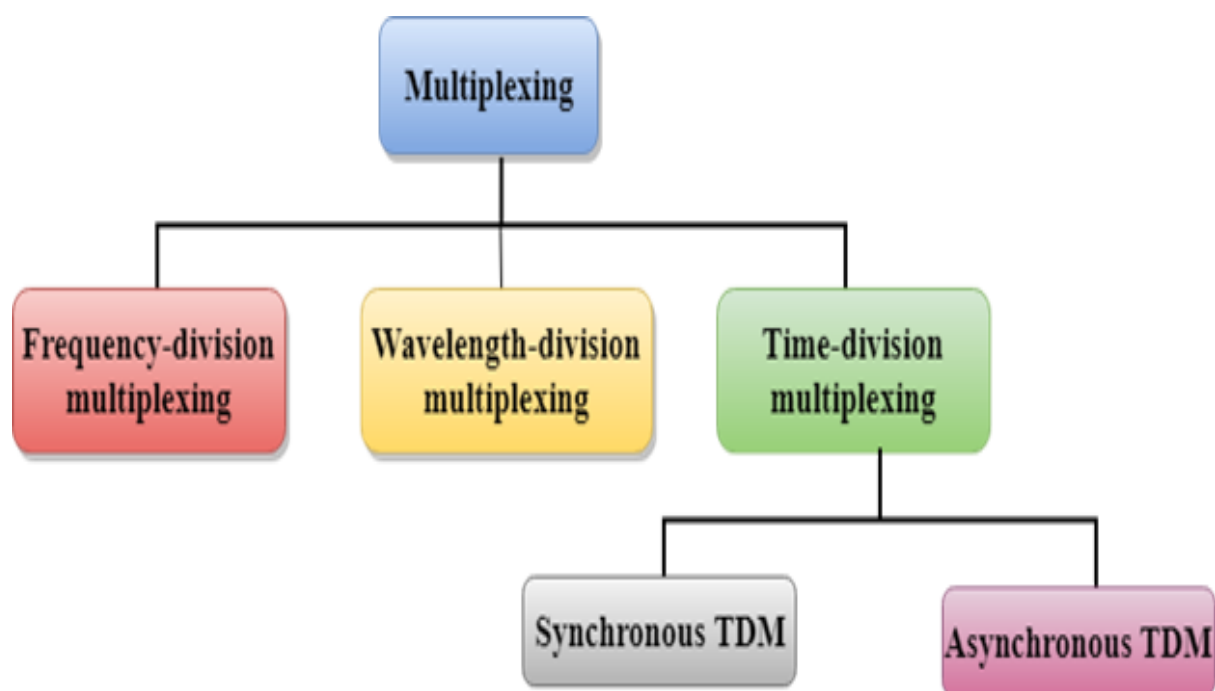
Host A wants to send 10 frames to Host B. How many frames are transmitted by Host A if every 5th frame transmitted by Host A is either corrupted or lost? What would be the number of transmissions in: Go-Back-3 ARQ?

MULTIPLEXING

- Multiplexing is a technique used to combine and send the multiple data streams over a single medium. The process of combining the data streams is known as multiplexing and hardware used for multiplexing is known as a multiplexer.
- Multiplexing is achieved by using a device called Multiplexer (MUX) that combines n input lines to generate a single output line.
- Multiplexing follows many-to-one, i.e., n input lines and one output line.
- Demultiplexing is achieved by using a device called Demultiplexer (DEMUX) available at the receiving end. DEMUX separates a signal into its component signals (one input and n outputs). Therefore, we can say that demultiplexing follows the one-to-many approach

Why Multiplexing?

- The transmission medium is used to send the signal from sender to receiver. The medium can only have one signal at a time.
- If there are multiple signals to share one medium, then the medium must be divided in such a way that each signal is given some portion of the available bandwidth. For example: If there are 10 signals and bandwidth of medium is 100 units, then the 10 unit is shared by each signal.
- When multiple signals share the common medium, there is a possibility of collision. Multiplexing concept is used to avoid such collision.
- Transmission services are very expensive.



Frequency Division Multiplexing (FDM)

Frequency Division Multiplexing (FDM) is a multiplexing technique in which the total available bandwidth of a communication channel is divided into multiple non-overlapping frequency bands, and each signal is transmitted simultaneously on a different frequency.

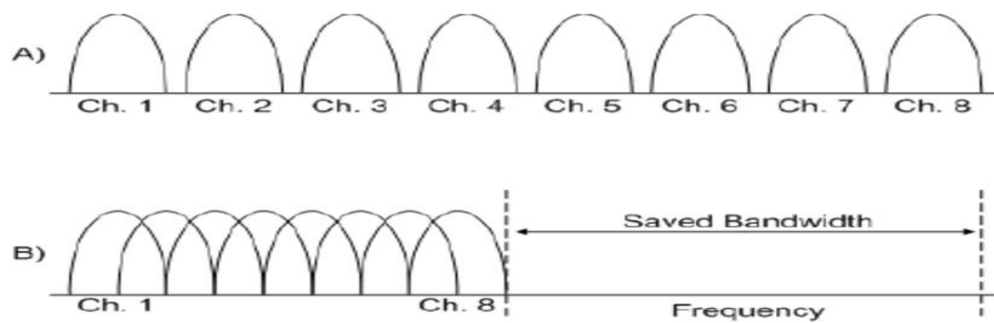
Basic Idea

Imagine a highway with many lanes.

Each car (signal) uses a different lane (frequency band).

All cars move at the same time without colliding because they stay in their own lane.

That's FDM.



What the diagram shows:

1. Multiple input signals
2. Each signal modulates a different carrier frequency
3. All modulated signals are combined
4. Combined signal transmitted over one channel
5. At receiver, filters separate each frequency

How FDM Works

1. **Input Signals**
Multiple analog signals are ready to be transmitted.
2. **Carrier Modulation**
Each signal is shifted to a different frequency using a modulator.
3. **Guard Bands Added**
Small unused frequency gaps are inserted between channels to avoid interference.
4. **Multiplexing**
All frequency-shifted signals are combined into one composite signal.
5. **Transmission**
The combined signal travels through a single medium.
6. **Demultiplexing**
At receiver, bandpass filters extract each frequency band.

A **guard band** is a small unused frequency range placed between adjacent channels.

Why needed?

Without guard bands:

- Signals may overlap
- Crosstalk occurs
- Data distortion happens

Guard bands prevent interference but slightly waste bandwidth.

Wavelength Division Multiplexing (WDM)

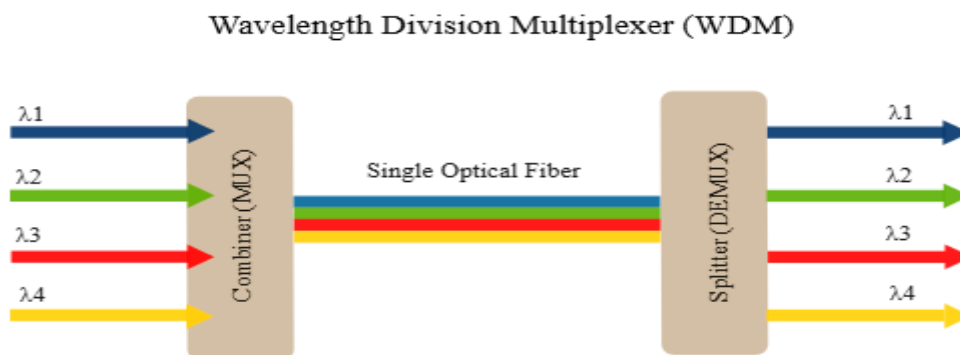
Wavelength Division Multiplexing (WDM) is a multiplexing technique used in **optical fiber communication**, where multiple signals are transmitted simultaneously over a single optical fiber using **different wavelengths (colors) of light**.

It is the **optical equivalent of FDM** (Frequency Division Multiplexing).

Basic Principle

- Optical fiber has extremely large bandwidth.
- Different light wavelengths carry different data signals.
- All wavelengths travel together through the same fiber.
- At the receiver, optical filters separate each wavelength

Working Process



Optical Signal Generation

Each data signal is converted into light using a **laser source**.

Different Wavelength Assignment

Each signal is assigned a unique wavelength ($\lambda_1, \lambda_2, \lambda_3, \dots$).

Multiplexing

An optical multiplexer combines all wavelengths into a single fiber.

Transmission

All wavelengths travel simultaneously through the fiber.

Demultiplexing

At the receiver:

- Optical filters separate wavelengths.
- Photodetectors convert light back to electrical signals.

Time Division Multiplexing (TDM)

Time Division Multiplexing (TDM) is a multiplexing technique in which multiple signals share a single communication channel by dividing the transmission time into **small time slots**.

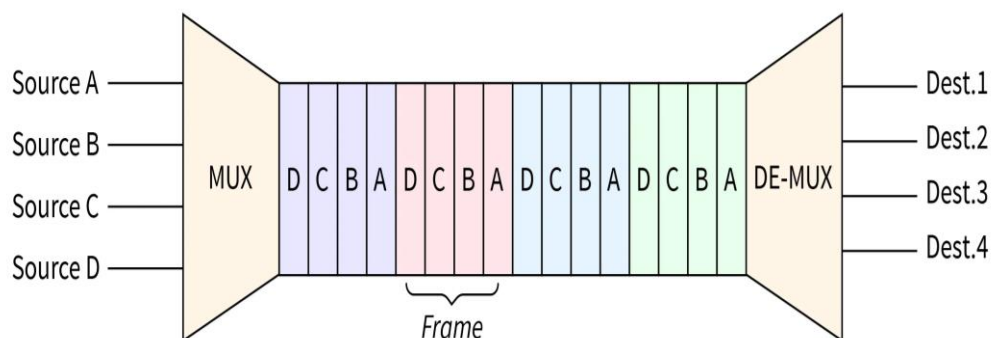
Each signal is transmitted one after another in a cyclic order, but so quickly that it appears simultaneous.

Mainly used in **digital communication systems**.

Basic Principle

- The entire bandwidth is used by each signal.
- But only for a **short duration (time slot)**.
- After one signal's slot ends, the next signal transmits.

Working Process



Input Signals

Multiple digital signals are connected to the multiplexer.

Time Slot Allocation

Each signal is assigned a fixed time slot within a frame.

Frame Formation

A complete cycle of time slots is called a **Frame**.

Example:

4 users → Frame = 4 time slots

User1 → User2 → User3 → User4 → Repeat

Transmission

The frame is transmitted over the channel.

Demultiplexing

At the receiver:

- Time slots are identified.
- Original signals are separated.

Types of TDM

Synchronous TDM

Characteristics

- Fixed time slots assigned to each user.
- Slots are reserved even if user has no data.
- Frame structure remains constant.

Example

If there are 5 users:

- Each frame always has 5 time slots.
- Even if only 2 users are active, 3 slots remain empty.

Advantages

- ✓ Simple implementation
- ✓ Easy synchronization

Disadvantages

- ✗ Bandwidth wastage (empty slots)
- ✗ Less efficient

Asynchronous TDM (Statistical TDM)

Characteristics

- Time slots assigned only to active users.
- No empty slots.
- Frame size may vary.

Example

If 5 users exist but only 2 active:

- Only 2 slots are used.
- No wastage.

Advantages

- ✓ Better bandwidth utilization
- ✓ Higher efficiency

Disadvantages

- ✗ More complex control mechanism
- ✗ Requires buffering